



Structured Outputs

Structured Outputs

1) What they are (and why they matter)

Structured outputs are responses returned in a **specific, machine-parseable format**—commonly JSON, XML, CSV, or key–value blocks—instead of free-form prose. They:

- **Impose order** on the model's answer (keys, types, sections).
- **Reduce hallucinations** by constraining what can be produced.
- **Enable automation** (easy parsing, validation, storage).
- **Integrate cleanly** with apps, APIs, and data pipelines.

| Mental model: Turn a chatty model into a predictable microservice.

2) Where they shine (and where they don't)

Great for

- Data extraction & classification
- Filling forms / creating records
- API calls & orchestrated agents (tool/function calling)
- Reports with fixed sections (dashboards, tickets, summaries)

Use cautiously

- Open-ended creative writing
 - Long narratives where rigid schemas may hinder flow
-

3) Control levels (from weakest to strongest)

1. Instruction-only

"Return JSON with fields A, B, C."

- Easiest to adopt, but may drift.

2. Provider features

- **JSON/grammar modes** (force valid JSON or a grammar).
- **Function/Tool calling** (model must output arguments to a schema).
- Most reliable when available.

3. External validation

- Parse → **validate** (JSON Schema / Zod / Pydantic / XSD).
- Reject/repair non-conforming outputs.

| Rule: Prefer function calling or JSON mode; always validate afterward.

4) Schema design tips

- **Stable keys** with clear semantics (avoid renaming between versions).
 - Use **enums** and **unions** for categorical fields.
 - Make optional fields **explicit** (nullable vs. omitted).
 - Keep **arrays** homogeneous; define item schemas.
 - Include a **version** field for evolution.
 - Add **id** or **source** for traceability.
-

5) Prompt patterns (copy-paste)

5.1 JSON-only contract (instruction-level)

```
You are a structured-output generator.  
Return ONLY valid JSON. Do not include code fences or commentary.
```

```
Type (TypeScript):  
type MovieClassification = {
```

```

title: string;
year: number;
genres: ("Action"|"Comedy"|"Drama"|"Horror"|"Sci-Fi"|"Romance")[];
rating: number; // 0–10
verdict: "Recommended" | "Skip";
reasons: string[]; // 1–5 concise bullets
source: { url: string; extractedAt: string }; // ISO-8601
schemaVersion: "1.0";
};

```

Task:

Classify the movie described below and fill all fields. If unknown, use null.

DATA:

<<<

[Paste movie description here]

>>>

Helpful params

- `temperature=0.2–0.5` (stability)
- `max_tokens` sized for the largest valid object
- `stop` sentinel (e.g., `###END###`) if not using JSON mode

5.2 Grammar/JSON mode (if the provider supports it)

- Turn on **JSON mode** or provide an **EBNF/BNF** grammar so the model *cannot* emit invalid structures.
- Keep schemas **small and strict**; the larger the grammar, the easier to violate at length.

5.3 Function/Tool calling (arguments only)

Intent: the model decides *what* tool to call and returns **typed arguments**—no prose.

- Define a tool `classify_movie(args: MovieClassificationInput)`
- The runtime validates, then your code executes the tool.

Benefits: near-100% structure adherence; built-in validation & routing.

6) Example: movie classification (end-to-end)

6.1 JSON Schema (validation)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "MovieClassification",
  "type": "object",
  "required": ["title", "year", "genres", "rating", "verdict", "reasons", "schemaVersion"],
  "properties": {
    "title": {"type": "string"},
    "year": {"type": "integer", "minimum": 1888, "maximum": 2100},
    "genres": {
      "type": "array",
      "items": {"enum": ["Action", "Comedy", "Drama", "Horror", "Sci-Fi", "Romance"]},
      "minItems": 1
    },
    "rating": {"type": "number", "minimum": 0, "maximum": 10},
    "verdict": {"enum": ["Recommended", "Skip"]},
    "reasons": {"type": "array", "items": {"type": "string"}, "minItems": 1, "maxItems": 5},
    "source": {
      "type": "object",
      "required": ["url", "extractedAt"],
      "properties": {
        "url": {"type": "string", "format": "uri"},
        "extractedAt": {"type": "string", "format": "date-time"}
      }
    },
    "schemaVersion": {"const": "1.0"}
  },
}
```

```
"additionalProperties":false
}
```

6.2 Prompt (JSON-only)

Return ONLY valid JSON that matches the MovieClassification schema above.
No preamble or explanation.

6.3 Post-processing checklist

- Parse → validate (JSON Schema).
- If invalid: request **repair** ("Return a corrected JSON that passes validation; do not change semantic content.").
- Store along with **schemaVersion** for compatibility.

7) Extraction pattern (common in production)

Prompt

Extract the following fields from the text and return ONLY valid JSON:

```
type Invoice = {
  invoiceNumber: string;
  issueDate: string; // YYYY-MM-DD
  total: number;
  currency: "USD"|"EUR"|"ARS";
  vendor: { name: string; taxId: string|null };
  items: { description: string; qty: number; price: number
}[];
  notes: string|null;
  schemaVersion: "1.0";
}
```

Text:

<<<

```
[Paste invoice text]
>>>
```

Tip: Add a **policy**: "If a field is not present, return null (do not infer)."

8) Guardrails & reliability

- **Stop sequences:** Use a unique sentinel if not in JSON/grammar mode.
- **Length control:** Size `max_tokens` comfortably; truncation breaks JSON.
- **Determinism:** Lower temperature; consider `seed` if supported.
- **Drop extras:** Strip code fences, markdown, trailing prose before parsing.
- **Escaping:** Watch for newline/quote escapes; prefer JSON over CSV for nested data.
- **Versioning:** Include `schemaVersion`; evolve with backward compatibility.

9) Testing & metrics

- **Golden prompts** → assert valid JSON and field presence.
- **Mutation tests** (noisy text, extra whitespace, missing fields).
- **Adherence rate:** % generations passing validation on first try.
- **Repair rate:** % fixed in one follow-up.
- **Latency & token cost:** track before/after adopting structure.

10) Troubleshooting

Symptom	Likely Cause	Fix
Extra prose around JSON	Instruction-only; model defaults to chat	Enable JSON/grammar mode or function calling; "NO preamble" + strip regex
Invalid/malformed JSON	Truncated output or high randomness	Lower temperature; raise <code>max_tokens</code> ; add stop; provide a minimal valid example
Hallucinated fields	Schema too open / vague task	Tighten schema (<code>additionalProperties:false</code>); add enums; clarify "null if unknown"

Symptom	Likely Cause	Fix
Wrong datatypes	Ambiguous formatting	Specify formats (ISO dates, currency enums); validate and coerce or reject
Overly long arrays	No bounds in schema	Add <code>maxItems</code> ; instruct limits in prompt

11) Quick templates

Strict JSON (no chatter)

Return ONLY valid JSON. No code fences, no commentary.
 If a value is unknown, use null. Do not invent data.
 Schema version: "1.0".

NDJSON for streams

Output one JSON object per line (NDJSON). No blank lines. Exactly 100 lines.

Two-stage (planner → filler)

Stage 1: Produce an outline as JSON (sections, word budgets).
 Stage 2: For each section, return JSON with {sectionId, content}.

12) Key takeaways

- **Structured outputs** turn LLM responses into **contracts** your systems can trust.
- Prefer **function/JSON modes + validation**; keep schemas **tight** and **versioned**.
- Pair with **stop sequences**, **max_tokens**, and **low temperature** for stability.
- Treat prompts and schemas like code: **test, monitor, evolve**.